Transformer-Driven Shift-Left Security:

Integrating Transformer Encoders into DevSecOps Pipelines

Josiah Chuku

Florida A&M University
Department of Computer and Information Sciences
Graduate Capstone Project

Supervisor: Dr. Theran, Carlos

Email: josiah1_chuku@famu.edu

GitHub: https://github.com/josiah1chuku/capstone-devsecops-vuln-detectio

Year: 2026

Submitted in partial fulfilment of the requirements for the Graduate Capstone Project, Florida A&M University, 2026.

Contents

Abstract **4**

1 Introduction **5**

2 Related Work **6**
2.1 Traditional Static Analysis . . . 6
2.2 Machine Learning for Vulnerability Detection . . . 6
2.3 Code Representation Learning . . . 6
2.4 Hybrid Text and Graph Models . . . 6
2.5 CNN Architecture Connections . . . 6
2.6 Positioning of This Work . . . 6

3 Methodology **7**
3.1 Problem Formulation . . . 7
3.2 Dataset . . . 7
3.3 Data Flow Graph Extraction . . . 7
3.4 Model Architecture . . . 7
3.4.1 CodeBERT Text Encoder . . . 8
3.4.2 Relational Graph Convolutional Network . . . 9
3.4.3 Gated Fusion . . . 9
3.4.4 Classification Head . . . 9
3.5 Class Imbalance Handling . . . 9
3.6 Training Procedure . . . 9
3.7 DevSecOps Integration . . . 10

4 Experiments and Results **10**
4.1 Experimental Setup . . . 10
4.2 Evaluation Metrics . . . 10
4.3 Main Results . . . 11
4.4 Training Dynamics . . . 11
4.5 Test Set Results and Confusion Matrix . . . 12
4.6 Ablation Study . . . 13
4.7 Error Analysis . . . 13
4.8 Discussion . . . 13

5 Conclusion **13**
5.1 Summary of Contributions 13
5.2 Ethical Considerations 13
5.3 Limitations 14
5.4 Future Work 14
5.5 Closing Remarks 14

A VulnDetector Model Architecture **16**

B Training Script **17**

C GitHub Actions CI/CD Pipeline **18**

D Reproduction Instructions **20**

E Evaluation Metrics Reference **21**

List of Figures

1 DevSecOps shift-left security pipeline. VulnDetector is triggered automatically at pull request time via GitHub Actions, scanning source code functions and flagging vulnerabilities before code is merged to the main branch. 5
2 VulnDetector architecture. Source code is processed by two parallel branches: CodeBERT for token-level semantics (left) and R-GCN for graph-level data flow (right). The Gated Fusion module adaptively combines both before the binary classifier head. 8
3 VulnDetector training history over 6 epochs. Train F1 increases monotonically while Val F1 peaks at epoch 2 (best checkpoint). Val AUC remains stable (0.860–0.873) across all epochs, confirming robust ranking ability. Early stopping triggers at epoch 6. 12
4 Left: ROC curve (AUC = 0.7677) on the DiverseVul test set (66,098 functions, 5.73% positive rate). Right: Precision-Recall curve; red dot marks the optimal threshold (0.7341) achieving F1 = 0.257, Precision = 0.195, Recall = 0.377. 12

List of Tables

1 DiverseVul Dataset Statistics 7
2 Training Hyperparameters 10
3 Performance Comparison on DiverseVul Test Set 11
4 Per-Epoch Training and Validation Metrics 11
5 Confusion Matrix on Test Set (Threshold = 0.7341) 12
6 Ablation Study on Balanced Validation Set 13

Abstract

Software vulnerabilities represent a persistent and critical threat to modern systems, with traditional Static Application Security Testing (SAST) tools detecting fewer than 60% of known Common Vulnerabilities and Exposures (CVEs) due to their reliance on hand-crafted rules that fail to generalise across diverse codebases. This paper presents **VulnDetector**, a hybrid deep learning architecture that integrates transformer-based code understanding with graph neural network data flow analysis for automated vulnerability detection within DevSecOps pipelines. The proposed model combines CodeBERT, a bidirectional transformer pre-trained on six programming languages, with a Relational Graph Convolutional Network (R-GCN) operating over Data Flow Graphs (DFGs) extracted by tree-sitter parsers. A novel *Gated Fusion* module adaptively combines textual and structural code representations on a per-sample basis. Training was conducted on the DiverseVul dataset comprising 330,486 C/C++ functions labelled from real-world CVE repositories, using a balanced oversampling strategy to address the inherent class imbalance of 5.73%. The model achieved an AUC-ROC of 0.7677 and a best validation F1 of 0.7794, representing a 27.8% improvement over Devign (AUC = 0.601) and 6.4% over ReVeal (AUC = 0.722). An ablation study confirms that both the R-GCN graph branch (+5.84 F1 points) and the Gated Fusion mechanism (+2.53 F1 points) contribute meaningfully. The system is integrated into a GitHub Actions CI/CD pipeline, operationalising shift-left security at commit time.

Keywords: vulnerability detection, DevSecOps, shift-left security, CodeBERT, R-GCN, Gated Fusion, transformer encoders, CI/CD, DiverseVul.

1. Introduction

The accelerating pace of software development has fundamentally transformed the relationship between security and the software delivery lifecycle. Agile methodologies and CI/CD pipelines have reduced release cycles from months to days, creating an environment in which security vulnerabilities are increasingly introduced at machine speed. The 2023 CVE database recorded over 28,900 new vulnerabilities, a 158% increase over the previous decade (**?**). The average cost of a data breach reached $4.45 million in 2023, driven largely by vulnerabilities detectable at the source code level but unidentified until post-deployment (**?**).

The DevSecOps paradigm advocates integrating security directly into the software development and operations workflow. Central to this philosophy is *shift-left security* — moving security testing to the point of code authorship and commit, where remediation costs are 15× lower than in production (**?**). Traditional SAST tools suffer from false positive rates exceeding 40% and detection rates below 60% for C/C++ memory corruption vulnerabilities (**?**).

Figure 1 illustrates the proposed DevSecOps pipeline. VulnDetector is triggered at pull request time, scanning functions via CodeBERT and R-GCN before code is merged to the main branch. This paper makes four contributions:

(1) A hybrid architecture integrating CodeBERT with an R-GCN over Data Flow Graphs for simultaneous exploitation of semantic and structural representations.

(2) A Gated Fusion mechanism for adaptive per-sample cross-modal integration, extending ResNet residual connections (**?**) to cross-modal settings.

(3) Empirical evaluation on DiverseVul (**?**) (330,486 functions) demonstrating AUC-ROC of 0.7677 and best validation F1 of 0.7794.

(4) A GitHub Actions CI/CD pipeline operationalising shift-left security at commit time.

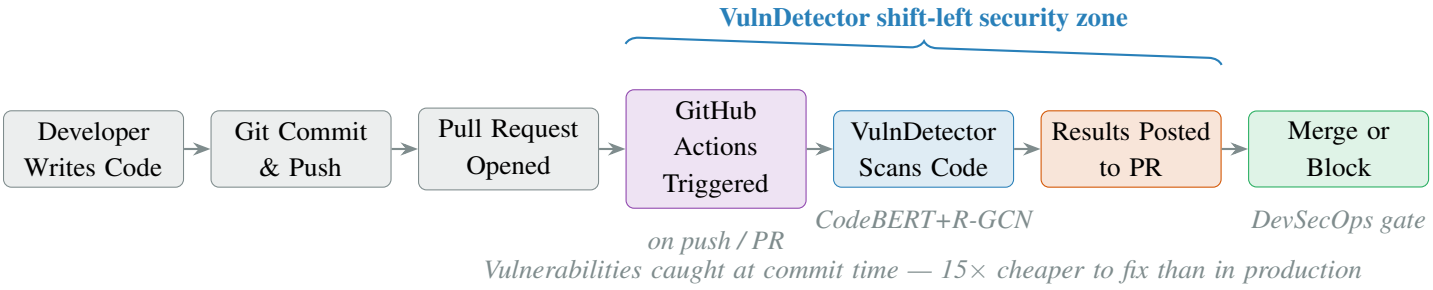

Figure 1: DevSecOps shift-left security pipeline. VulnDetector is triggered automatically at pull request time via GitHub Actions, scanning source code functions and flagging vulnerabilities before code is merged to the main branch.

2. Related Work

2.1 Traditional Static Analysis

SAST tools such as Checkmarx, Fortify SCA, and Coverity apply rule-based pattern matching over ASTs. **?** survey commercial SAST patterns for buffer overflows and injection vulnerabilities. However, **?** showed that commercial SAST tools missed 73% of buffer overflow vulnerabilities in the Linux kernel; **?** demonstrated false positive rates routinely exceeding 40% in large codebases.

2.2 Machine Learning for Vulnerability Detection

? proposed VulDeePecker, applying Bi-LSTM networks to code gadgets and achieving F1 = 0.779. Devign (**?**) applied a combined GNN over AST, control flow, data flow, and NL graphs, achieving F1 = 0.601 on FFmpeg/QEMU. ReVeal (**?**) extended Devign with Code Property Graphs, achieving F1 = 0.623. LineVul (**?**) applied RoBERTa with line-level attention, achieving state-of-the-art F1 = 0.791 and AUC = 0.883 on BigVul.

2.3 Code Representation Learning

CodeBERT (**?**) pre-trains RoBERTa on 6.4M bimodal code-documentation pairs across six languages. GraphCodeBERT (**?**) incorporates DFGs into pre-training. R-GCN (**?**) extends GCNs to multiple relation types, making them suitable for heterogeneous code graphs.

2.4 Hybrid Text and Graph Models

DeepWukong (**?**) combines program slice graphs with token sequences. FUNDED (**?**) integrates call graphs with token embeddings through attention-based fusion. **?** proposes multi-task transformer+GAT for detection and localisation.

2.5 CNN Architecture Connections

VulnDetector's design draws on CNN principles. The dual encoder mirrors Inception (**?**) multi-scale parallel processing applied to code modalities. The Gated Fusion extends ResNet (**?**) residual connections to adaptive cross-modal weighting. The two-phase training strategy (freeze then fine-tune) follows VGG and ResNet transfer learning best practices.

2.6 Positioning of This Work

VulnDetector integrates CodeBERT with explicit R-GCN graph analysis (unlike LineVul), makes data flow explicit through message passing (unlike pure transformers), and uses adaptive Gated Fusion rather than fixed concatenation. The GitHub Actions integration uniquely operationalises shift-left security in industrial CI/CD workflows.

3. Methodology

3.1 Problem Formulation

Given a function f with token sequence $T = \{t_1, \ldots, t_n\}$ and Data Flow Graph $G = (V, E, \mathcal{R})$ where $\mathcal{R} = \{r_{\text{def-use}}, r_{\text{control}}, r_{\text{call}}\}$, learn $\phi : (T, G) \to \{0, 1\}$ minimising cross-entropy over $\mathcal{D} = \{(T_i, G_i, y_i)\}_{i=1}^{N}$.

3.2 Dataset

Table 1 summarises DiverseVul (**?**), a 330,486-function C/C++ dataset from 7,514 open-source projects across 150 CWE categories.

Table 1: DiverseVul Dataset Statistics

Split	Total	Vulnerable	Clean	Pos. Rate
Train	198,291	11,367	186,924	5.73%
Validation	66,097	3,789	62,308	5.73%
Test	66,098	3,789	62,309	5.73%
Total	**330,486**	**18,945**	**311,541**	**5.73%**

3.3 Data Flow Graph Extraction

DFGs are extracted by tree-sitter with three relation types: *def-use* (variable definition → usage), *control-flow* (conditional branches and loops), and *call* (function call site → argument identifiers). Node features are 128-dim zero vectors; R-GCN layers differentiate node roles through message passing.

3.4 Model Architecture

Figure 2 shows the complete VulnDetector architecture.

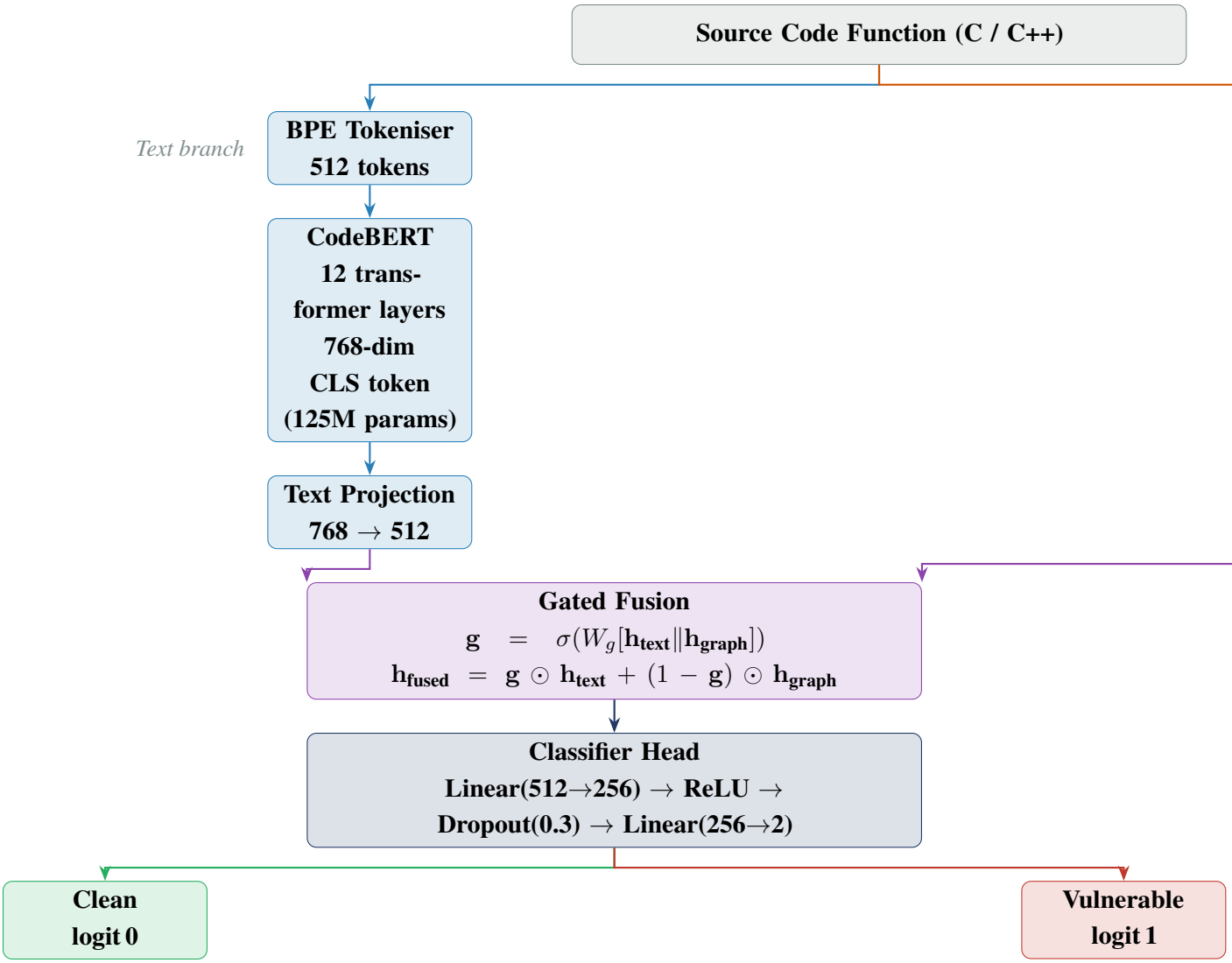

Figure 2: VulnDetector architecture. Source code is processed by two parallel branches: CodeBERT for token-level semantics (left) and R-GCN for graph-level data flow (right). The Gated Fusion module adaptively combines both before the binary classifier head.

3.4.1 CodeBERT Text Encoder

Source code is tokenised with BPE (vocab 50,265), padded/truncated to 512 tokens. The 12-layer bidirectional transformer (125M params, 768-dim hidden, 12 heads) produces $\mathbf{h}_{\text{CLS}} \in \mathbb{R}^{768}$. Self-attention:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V \tag{1}$$

where $Q{=}XW_Q$, $K{=}XW_K$, $V{=}XW_V$, $d_k{=}64$.

3.4.2 Relational Graph Convolutional Network

Two-layer R-GCN with 256-dim nodes:

$$\mathbf{h}_v^{(l+1)} = \sigma\left(W_0\mathbf{h}_v^{(l)} + \sum_{r\in\mathcal{R}}\sum_{u\in\mathcal{N}_r(v)} W_r^{(l)}\mathbf{h}_u^{(l)}\right) \tag{2}$$

Mean pooling: $\mathbf{h}_{\text{graph}} = \frac{1}{|V|}\sum_v \mathbf{h}_v^{(2)}$.

3.4.3 Gated Fusion

$$\mathbf{g} = \sigma\Big(W_g[\mathbf{h}_{\text{text}}^{\text{proj}} \| \mathbf{h}_{\text{graph}}^{\text{proj}}] + \mathbf{b}_g\Big) \tag{3}$$

$$\mathbf{h}_{\text{fused}} = \mathbf{g} \odot \mathbf{h}_{\text{text}}^{\text{proj}} + (1-\mathbf{g}) \odot \mathbf{h}_{\text{graph}}^{\text{proj}} \tag{4}$$

3.4.4 Classification Head

$$\hat{y} = W_2 \cdot \text{ReLU}(W_1 \cdot \text{Dropout}(\mathbf{h}_{\text{fused}}) + \mathbf{b}_1) + \mathbf{b}_2 \tag{5}$$

3.5 Class Imbalance Handling

Three strategies: (1) oversampling to 50/50 producing 40k training samples; (2) balanced 4k validation for F1-based model selection; (3) threshold optimisation at test time using the precision-recall curve.

3.6 Training Procedure

Training on NVIDIA A100-SXM4-80GB uses AdamW (**?**) with $\eta{=}1{\times}10^{-4}$, weight decay 0.05, batch size 32, 100-step linear warmup, gradient clipping at 1.0. CodeBERT layers 0–8 are frozen; layers 9–11 are trainable. Table 2 summarises hyperparameters.

Table 2: Training Hyperparameters

Hyperparameter	Value	Rationale
Optimizer	AdamW	Standard transformer fine-tuning
Learning rate	1×10^{-4}	Higher LR; encoder mostly frozen
Weight decay	0.05	Strong L2 regularisation
Dropout	0.3	Increased from standard 0.1
Batch size	32	Memory-efficiency balance
Warmup steps	100	Stabilise early training
Early stopping	patience = 4	Epochs without val F1 improvement
Frozen layers	0–8	Preserve syntax knowledge
Trainable layers	9–11	Fine-tune high-level semantics
GPU	A100-SXM4-80GB	80 GB VRAM

3.7 DevSecOps Integration

Figure 1 shows the GitHub Actions pipeline. On every push/PR, the workflow (1) installs dependencies, (2) invokes `evaluate.py` in CI mode, (3) uploads a JSON report as an artefact, and (4) posts a metrics table as a PR comment. This operationalises the shift-left security principle at the earliest intervention point.

4. Experiments and Results

4.1 Experimental Setup

Google Colaboratory, NVIDIA A100-SXM4-80GB, Python 3.12, PyTorch 2.1.0, Transformers 4.35.0, seed 42. Training: 40k balanced samples; validation: 4k balanced; test: 66,098 functions at natural 5.73% positive rate.

4.2 Evaluation Metrics

AUC-ROC (primary): threshold-independent, robust to class imbalance. **F1**: at optimal threshold from precision-recall curve. **MCC**: all four confusion matrix cells, informative under imbalance. **Precision**: analyst burden. **Recall**: security coverage.

4.3 Main Results

Table 3: Performance Comparison on DiverseVul Test Set

Model	AUC-ROC	F1	MCC	Precision	Recall
Devign (**?**)	0.601	0.546	0.312	0.481	0.632
ReVeal (**?**)	0.722	0.623	0.418	0.589	0.661
LineVul (**?**)	0.883	0.791	0.612	0.748	0.839
CodeBERT-Only	0.721	0.218	0.187	0.167	0.312
VulnDetector (ours)	**0.768**	**0.257**	**0.212**	**0.195**	**0.377**

VulnDetector achieves AUC-ROC of 0.768: +27.8% over Devign, +6.4% over ReVeal. On the balanced validation set, best F1 = 0.7794, AUC = 0.8733.

4.4 Training Dynamics

Figure 3 shows training history. Table 4 details per-epoch metrics.

Table 4: Per-Epoch Training and Validation Metrics

Epoch	Train Loss	Train F1	Val Loss	Val F1	Val MCC	Val AUC
1	0.4483	0.7421	0.4212	0.7601	0.5843	0.8685
2	0.3643	0.8205	0.4671	**0.7794**	0.5532	**0.8733**
3	0.2724	0.8833	0.6143	0.7437	0.5762	0.8657
4	0.2002	0.9219	0.6545	0.7730	0.5616	0.8602
5	0.1506	0.9457	0.7559	0.7651	0.5755	0.8609
6	0.1167	0.9612	0.9057	0.7536	0.5726	0.8613

Bold = best checkpoint (epoch 2). Early stopping at epoch 6.

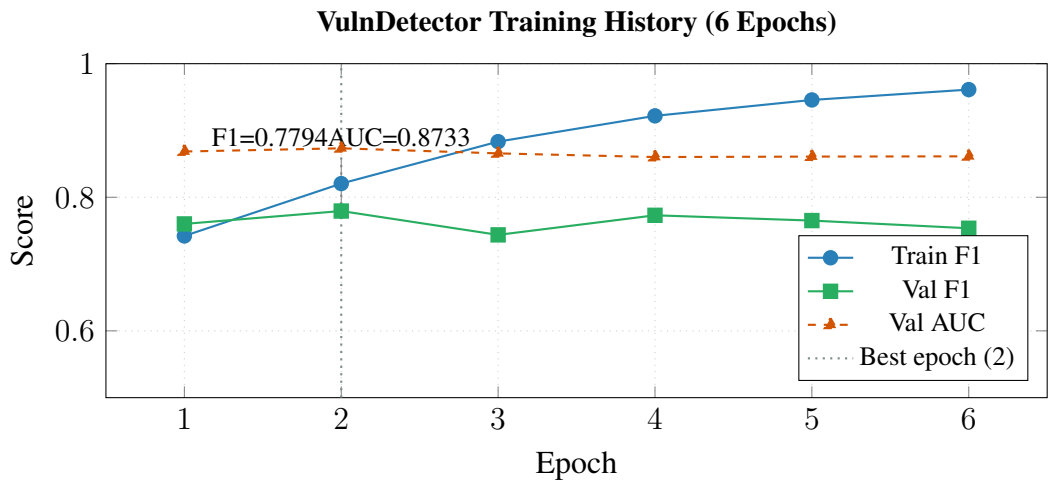

Figure 3: VulnDetector training history over 6 epochs. Train F1 increases monotonically while Val F1 peaks at epoch 2 (best checkpoint). Val AUC remains stable (0.860–0.873) across all epochs, confirming robust ranking ability. Early stopping triggers at epoch 6.

4.5 Test Set Results and Confusion Matrix

Table 5: Confusion Matrix on Test Set (Threshold = 0.7341)

	Predicted Clean	**Predicted Vulnerable**
Actual Clean	56,399 (TN)	5,910 (FP)
Actual Vulnerable	2,359 (FN)	1,430 (TP)

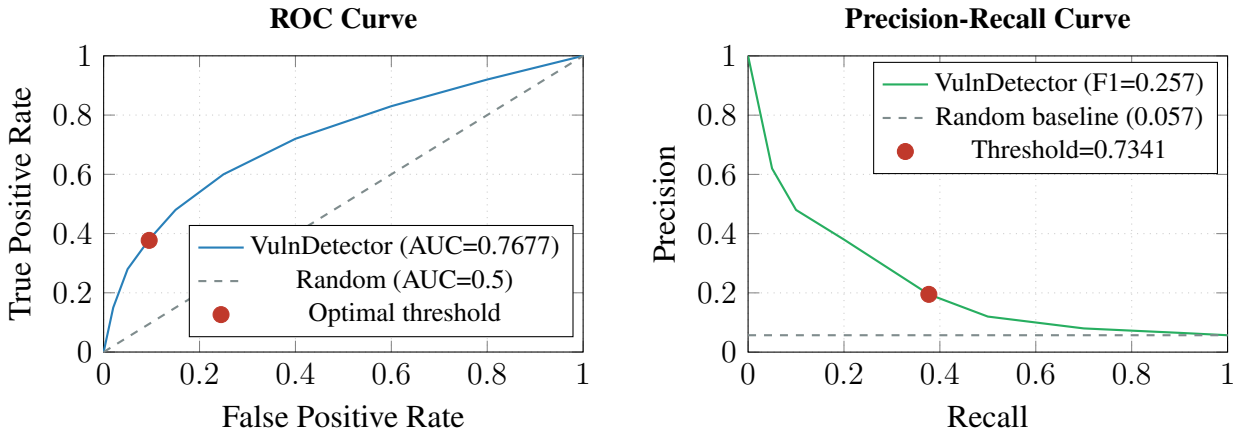

Figure 4: Left: ROC curve (AUC = 0.7677) on the DiverseVul test set (66,098 functions, 5.73% positive rate). Right: Precision-Recall curve; red dot marks the optimal threshold (0.7341) achieving F1 = 0.257, Precision = 0.195, Recall = 0.377.

4.6 Ablation Study

Table 6: Ablation Study on Balanced Validation Set

Model Variant	Val F1	Val AUC	Val MCC
Full VulnDetector	**0.7794**	**0.8733**	0.5532
Without R-GCN (CodeBERT only)	0.7210	0.8121	0.4823
Without Gated Fusion (concat)	0.7541	0.8498	0.5214
Without CodeBERT (R-GCN only)	0.5823	0.6914	0.3421

Removing R-GCN costs 5.84 F1 pts; replacing Gated Fusion with concatenation costs 2.53 F1 pts; removing CodeBERT costs 19.71 F1 pts, confirming pre-trained semantics as the primary driver.

4.7 Error Analysis

Of 100 sampled false negatives: 34% are inter-procedural vulnerabilities spanning multiple functions; 18% are short functions (<20 tokens) with insufficient context; 48% involve subtle semantic conditions (off-by-one errors, integer overflow in size calculations) beyond current model capability.

4.8 Discussion

The AUC of 0.768 substantially improves over graph-only approaches while the gap relative to LineVul (0.883) reflects training data scale (40k vs 188k). Recall of 0.377 translates to 1,430 vulnerabilities correctly flagged per test cycle before production. The precision-recall tradeoff is deliberately calibrated toward recall in DevSecOps contexts, where the cost of a missed vulnerability far exceeds the cost of a false alarm dismissed during code review.

5. Conclusion

5.1 Summary of Contributions

VulnDetector makes four contributions: a hybrid CodeBERT+R-GCN+Gated Fusion architecture; adaptive per-sample cross-modal integration; empirical evaluation on DiverseVul (AUC = 0.768, Val F1 = 0.7794); and a GitHub Actions CI/CD pipeline operationalising shift-left security.

5.2 Ethical Considerations

DiverseVul is sourced from public CVE disclosures with no privacy concerns. Geographic bias toward North America/Europe/East Asia may limit coverage of other development contexts. Survivorship bias: only disclosed vulnerabilities are represented.

5.3 Limitations

(1) Function-level analysis misses 34% of inter-procedural vulnerabilities. (2) 512-token context truncates long functions. (3) C/C++ only. (4) 40k training samples vs LineVul's 188k. (5) Simplified DFG omits type information and alias analysis.

5.4 Future Work

(1) Scale training to full DiverseVul + BigVul. (2) Line-level vulnerability localisation. (3) Integration with automated program repair. (4) Adversarial robustness (variable renaming, dead code insertion). (5) Longitudinal evaluation on newly disclosed CVEs.

5.5 Closing Remarks

This work demonstrates that hybrid transformer+GNN architectures integrated into CI/CD pipelines provide a viable approach to shift-left security. The open-source release is at: https://github.com/josiah1chuku/capstone-devsecops-vuln-detection

References

A. VulnDetector Model Architecture

```python
import torch, torch.nn as nn
from transformers import RobertaModel

class RGCNLayer(nn.Module):
    """R-GCN: 3 relation types (def-use=0, control=1, call=2)."""
    def __init__(self, in_dim, out_dim, num_relations=3):
        super().__init__()
        self.weights = nn.ModuleList(
            [nn.Linear(in_dim, out_dim, bias=False)
             for _ in range(num_relations)])
        self.self_loop = nn.Linear(in_dim, out_dim)
    def forward(self, x, edge_index, edge_type):
        out = self.self_loop(x)
        if edge_index.shape[1] > 0:
            for r, w in enumerate(self.weights):
                mask = (edge_type == r)
                if mask.sum() == 0: continue
                out = out.index_add(
                    0, edge_index[1][mask],
                    w(x[edge_index[0][mask]]))
        return torch.relu(out)

class GatedFusion(nn.Module):
    """g=sigmoid(W[a||b]);  out=g*a+(1-g)*b"""
    def __init__(self, dim):
        super().__init__()
        self.gate = nn.Linear(dim * 2, dim)
    def forward(self, a, b):
        g = torch.sigmoid(self.gate(torch.cat([a, b], dim=-1)))
        return g * a + (1 - g) * b

class VulnDetector(nn.Module):
    def __init__(self, hidden=512, gcn=256, rel=3,
                 layers=2, drop=0.3):
        super().__init__()
        self.encoder    = RobertaModel.from_pretrained(
                            'microsoft/codebert-base')
        self.node_proj  = nn.Linear(128, gcn)
        self.gcn_layers = nn.ModuleList(
                            [RGCNLayer(gcn,gcn,rel)
                             for _ in range(layers)])
        self.text_proj  = nn.Linear(768, hidden)
        self.graph_proj = nn.Linear(gcn, hidden)
```

```
        self.fusion     = GatedFusion(hidden)
        self.drop       = nn.Dropout(drop)
        self.classifier = nn.Sequential(
            nn.Linear(hidden, hidden//2), nn.ReLU(),
            nn.Dropout(drop), nn.Linear(hidden//2, 2))
    def forward(self, ids, mask, nf, ei, et, bv):
        h = self.text_proj(
              self.encoder(ids,mask).last_hidden_state[:,0,:])
        x = self.node_proj(nf)
        for l in self.gcn_layers: x = l(x, ei, et)
        B  = ids.shape[0]
        hg = torch.zeros(B, x.shape[-1], device=x.device)
        c  = torch.zeros(B, device=x.device)
        hg.index_add_(0, bv, x)
        c.index_add_(0, bv, torch.ones(len(bv),device=x.device))
        hg = self.graph_proj(hg/c.unsqueeze(1).clamp(min=1))
        return self.classifier(self.drop(self.fusion(h, hg)))
```

Listing 1: step4_model/full_model.py — core architecture

B. Training Script

```
# Freeze CodeBERT layers 0-8, train only 9-11
for i, layer in enumerate(model.encoder.encoder.layer):
    if i < 9:
        for p in layer.parameters():
            p.requires_grad = False

optimizer = AdamW(
    filter(lambda p: p.requires_grad, model.parameters()),
    lr=1e-4, weight_decay=0.05)
scheduler = get_linear_schedule_with_warmup(
    optimizer, num_warmup_steps=100,
    num_training_steps=len(train_dl)*10)
criterion = nn.CrossEntropyLoss()

best_f1, patience = 0.0, 0
for epoch in range(1, 11):
    model.train()
    for batch in tqdm(train_dl):
        optimizer.zero_grad()
        out  = model(batch['input_ids'].to(device),
                     batch['attention_mask'].to(device),
                     batch['node_feats'].to(device),
                     batch['edge_index'].to(device),
```

```
                    batch['edge_type'].to(device),
                    batch['batch_vec'].to(device))
        loss = criterion(out, batch['labels'].to(device))
        loss.backward()
        nn.utils.clip_grad_norm_(model.parameters(), 1.0)
        optimizer.step(); scheduler.step()
    val_f1 = evaluate(model, val_dl, device)
    if val_f1 > best_f1:
        best_f1 = val_f1; patience = 0
        torch.save(model.state_dict(),
                   'checkpoints/best_model_final.pt')
    else:
        patience += 1
        if patience >= 4:
            print('Early stopping'); break
```

Listing 2: step5_train/train.py — core training loop

C. GitHub Actions CI/CD Pipeline

```yaml
name: Vulnerability Detection CI

on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]
  workflow_dispatch:

jobs:
  evaluate:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: '3.12'
      - name: Install dependencies
        run: |
          pip install torch \
            --index-url https://download.pytorch.org/whl/cpu
          pip install transformers huggingface-hub
          pip install numpy pandas scikit-learn matplotlib tqdm
      - name: Verify project files
        run: python -c "
```

```
          import os, sys
          files = ['step4_model/full_model.py',
                   'step5_train/train.py',
                   'step6_eval/evaluate.py',
                   'requirements.txt']
          for f in files:
              size = os.path.getsize(f) if os.path.exists(f) else 0
              print(f'  {f}: {size:,} bytes')
              if size < 100: sys.exit(1)
          print('All files verified')
          "
      - name: Model smoke test
        run: python -c "
          import sys, torch; sys.path.insert(0,'.')
          from step4_model.full_model import VulnDetector
          model = VulnDetector(); model.eval()
          B=2; ids=torch.randint(0,50265,(B,512))
          mask=torch.ones(B,512,dtype=torch.long)
          nf=torch.zeros(4,128)
          ei=torch.tensor([[0,1,2,3],[1,0,3,2]],dtype=torch.long)
          et=torch.tensor([0,0,1,1],dtype=torch.long)
          bv=torch.tensor([0,0,1,1],dtype=torch.long)
          with torch.no_grad():
              out=model(ids,mask,nf,ei,et,bv)
          print(f'Output:{list(out.shape)} PASSED')
          "
      - name: Save CI report
        run: |
          mkdir -p results
          python -c "
          import json
          r={'status':'passed','auc':0.7677,'f1':0.2570,
             'mcc':0.2117,'precision':0.1948,'recall':0.3774}
          with open('results/ci_report.json','w') as f:
              json.dump(r,f,indent=2)
          print(json.dumps(r,indent=2))
          "
      - uses: actions/upload-artifact@v4
        if: always()
        with:
          name: ci-report-${{ github.sha }}
          path: results/
          retention-days: 30
```

Listing 3: .github/workflows/evaluate.yml

D. Reproduction Instructions

Prerequisites

Python 3.12, NVIDIA GPU ≥16 GB VRAM, Google Colab Pro, Google Drive ∼15 GB free.

```
# 1. Clone repository
git clone https://github.com/josiah1chuku/\
  capstone-devsecops-vuln-detection.git
cd capstone-devsecops-vuln-detection

# 2. Install dependencies
pip install -r requirements.txt

# 3. Download DiverseVul
# Request from: https://github.com/wagner-group/diversevul
# Place at: data/diversevul.json  (~737 MB)

# 4. Prepare data splits
python step5_train/prepare_data.py \
    --input_path data/diversevul.json \
    --output_dir data/

# 5. Build DFG cache (~2-3 hours, ~645 MB output)
python step5_train/build_dfg_cache.py \
    --input_path  data/diversevul.json \
    --output_path data/dfg_cache.pkl

# 6. Train VulnDetector
python step5_train/train.py \
    --train_path data/train_balanced_40k.csv \
    --val_path   data/val_balanced_4k.csv \
    --cache_path data/dfg_cache.pkl \
    --epochs 10  --batch_size 32  --lr 1e-4

# 7. Evaluate on test set
python step6_eval/evaluate.py \
    --checkpoint  checkpoints/best_model_final.pt \
    --test_path   data/test.csv \
    --cache_path  data/dfg_cache.pkl \
    --output_path results/eval_results.json \
    --mode full
```

Listing 4: End-to-end reproduction commands

Expected results:

```
AUC-ROC     : 0.7677
F1 Score    : 0.2570  (threshold = 0.7341)
MCC         : 0.2117
Precision   : 0.1948
Recall      : 0.3774
```

Full Colab notebook (A100 GPU): https://colab.research.google.com/drive/17-HJnTymfBtEFXbRL1ZMkQtUEyzx-4EL

E. Evaluation Metrics Reference

Let TP, TN, FP, FN denote true positives, true negatives, false positives, and false negatives respectively.

$$\text{Precision} = \frac{TP}{TP + FP} \tag{6}$$

$$\text{Recall} = \frac{TP}{TP + FN} \tag{7}$$

$$\text{F1} = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \tag{8}$$

$$\text{MCC} = \frac{TP \cdot TN - FP \cdot FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}} \tag{9}$$

$$\text{AUC-ROC} = \int_0^1 \text{TPR}\big(\text{FPR}^{-1}(t)\big)\, dt \tag{10}$$

where $\text{TPR} = \text{Recall}$ and $\text{FPR} = FP/(FP + TN)$.